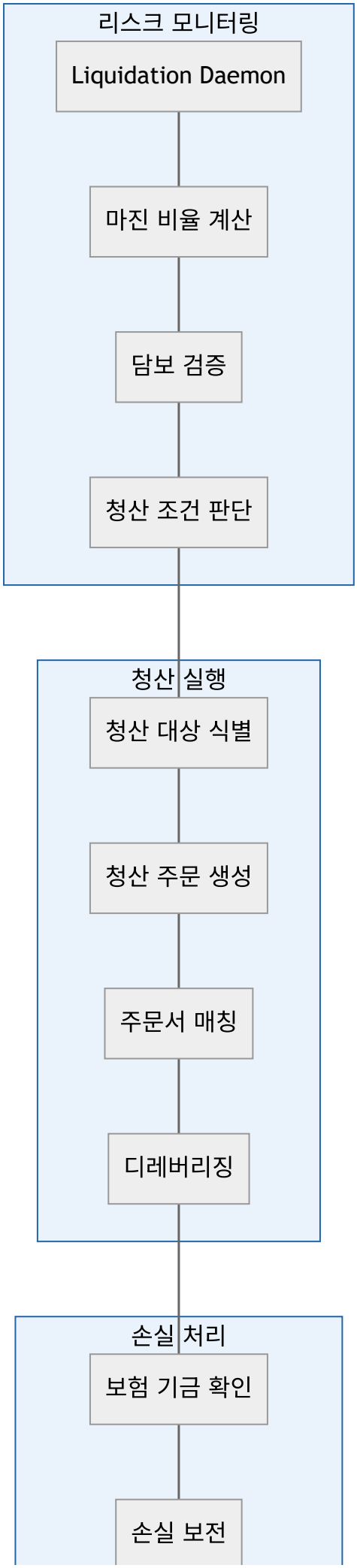
dYdX v4 거래 처리 플로우 - 리스크 관리 및 청산 프로세스 분석

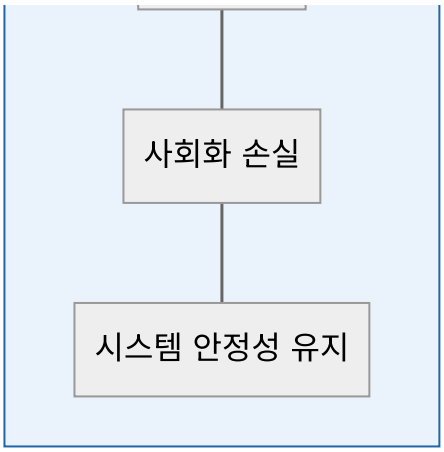
개요

이 문서는 dYdX v4의 리스크 관리 및 청산 프로세스를 상세히 분석합니다. 마진 비율 모니터링 및 청산 조건 판단 로직, 청산 주문 생성 및 실행 프로세스, 그리고 보험 기금 운용 및 손실 처리 메커니즘을 포함합니다.

리스크 관리 시스템 아키텍처

전체 리스크 관리 플로우


리스크 모니터링
Liquidation Daemon
마진 비율 계산
담보 검증
청산 조건 판단
청산 실행
청산 대상 식별
청산 주문 생성
주문서 매칭
디레버리징
손실 처리
보험 기금 확인
손실 보전



마진 비율 모니터링 및 청산 조건 판단 로직

1. 마진 계산 시스템

마진 타입 정의

```
type MarginRequirements struct {
// 초기 마진 (Initial Margin) - 새 포지션 개설 시 필요
InitialMargin *big.Int
// 유지 마진 (Maintenance Margin) - 포지션 유지를 위한 최소 마진
MaintenanceMargin *big.Int
// 순 담보 (Net Collateral) - 실제 보유 담보
NetCollateral *big.Int
}

type LiquidityTier struct {
Id uint32 // 유동성 계층 ID
Name string // 계층 이름 (예: "Large-Cap", "Mid-Cap")
InitialMarginPpm uint32 // 초기 마진 비율 (ppm 단위)
```

```
MaintenanceMarginPpm uint32 // 유지 마진 비율 (ppm 단위)

BasePositionNotional uint64 // 기본 포지션 명목가치

ImpactNotional uint64 // 임팩트 명목가치

}
```

유동성 계층별 마진 정책

```
// 예시: 다양한 자산의 유동성 계층

var LiquidityTiers = map[string]LiquidityTier{

"BTC": {

Id: 0,

Name: "Large-Cap",

InitialMarginPpm: 50000, // 5%

MaintenanceMarginPpm: 30000, // 3%

BasePositionNotional: 1000000, // $1M

ImpactNotional: 500000, // $500K

},

"ETH": {

Id: 1,

Name: "Large-Cap",

InitialMarginPpm: 60000, // 6%

MaintenanceMarginPpm: 35000, // 3.5%

BasePositionNotional: 1000000,

ImpactNotional: 500000,
```

```
},
"ALTCOIN": {
Id: 2,
Name: "Mid-Cap",
InitialMarginPpm: 100000, // 10%
MaintenanceMarginPpm: 70000, // 7%
BasePositionNotional: 100000, // $100K
ImpactNotional: 50000, // $50K
},
}
```

2. 담보 및 마진 계산 로직

순 담보 계산

```
func (k Keeper) GetNetCollateralAndMarginRequirements(
ctx  sdk.Context,
update  types.Update,
) (
bigNetCollateral *big.Int,
bigInitialMargin *big.Int,
bigMaintenanceMargin *big.Int,
err  error,
) {
```

```
subaccount := k.GetSubaccount(ctx, update.SubaccountId)

// 펀딩 정산이 포함된 서브계정 상태 계산

settledSubaccount, _, err := k.getSettledSubaccount(ctx, subaccount, true)

if err != nil {

return nil, nil, nil, err

}

// 업데이트 적용

updatedSubaccount := k.applyUpdatesToSubaccount(settledSubaccount, update)

bigNetCollateral = big.NewInt(0)

bigInitialMargin = big.NewInt(0)

bigMaintenanceMargin = big.NewInt(0)

// 1. 자산 포지션의 순 담보 계산

for _, assetPosition := range updatedSubaccount.AssetPositions {

if assetPosition.AssetId == assettypes.AssetUsdc.Id {

// USDC 잔고를 순 담보에 추가

bigNetCollateral.Add(bigNetCollateral, assetPosition.GetBigQuantums())

}

// TODO: 다중 담보 지원 시 다른 자산들도 처리

}

// 2. 영구 선물 포지션의 순 담보 및 마진 요구사항 계산

for _, perpetualPosition := range updatedSubaccount.PerpetualPositions {

perpetual, err := k.perpetualsKeeper.GetPerpetual(ctx, perpetualPosition.Perpetu

if err != nil {
```

```
return  nil, nil, nil, err
}
marketPrice, err := k.pricesKeeper.GetMarketPrice(ctx, perpetual.Params.MarketId
if  err != nil {
return  nil, nil, nil, err
}
liquidityTier, err := k.perpetualsKeeper.GetLiquidityTier(ctx, perpetual.Params.
if  err != nil {
return  nil, nil, nil, err
}
bigQuantums := perpetualPosition.GetBigQuantums()
// 포지션의 순 담보 계산 (미실현 손익 포함)
bigNetCollateralQuoteQuantums := perpkeeper.GetNetNotionalInQuoteQuantums(
perpetual,
marketPrice,
bigQuantums,
)
bigNetCollateral.Add(bigNetCollateral, bigNetCollateralQuoteQuantums)
// 마진 요구사항 계산
bigInitialMarginQuoteQuantums, bigMaintenanceMarginQuoteQuantums :=
perpkeeper.GetMarginRequirementsInQuoteQuantums(
perpetual,
marketPrice,
```

```
liquidityTier,
bigQuantums,
)
bigInitialMargin.Add(bigInitialMargin, bigInitialMarginQuoteQuantums)
bigMaintenanceMargin.Add(bigMaintenanceMargin, bigMaintenanceMarginQuoteQuantums
}
return  bigNetCollateral, bigInitialMargin, bigMaintenanceMargin, nil
}
```

포지션별 마진 계산

```
func  GetMarginRequirementsInQuoteQuantums(
perpetual  perptypes.Perpetual,
marketPrice  pricestypes.MarketPrice,
liquidityTier  perptypes.LiquidityTier,
bigQuantums *big.Int,
) (bigInitialMarginQuoteQuantums *big.Int, bigMaintenanceMarginQuoteQuantums *bi
// 포지션의 명목 가치 계산
bigNotionalQuoteQuantums := GetNetNotionalInQuoteQuantums(perpetual, marketPrice
bigAbsNotionalQuoteQuantums := new(big.Int).Abs(bigNotionalQuoteQuantums)
// 기본 마진 계산
bigInitialMarginQuoteQuantums = lib.BigIntMulPpm(
bigAbsNotionalQuoteQuantums,
```

```
liquidityTier.InitialMarginPpm,
)
bigMaintenanceMarginQuoteQuantums = lib.BigIntMulPpm(
bigAbsNotionalQuoteQuantums,
liquidityTier.MaintenanceMarginPpm,
)
// 대형 포지션에 대한 추가 마진 요구사항 적용
if  bigAbsNotionalQuoteQuantums.Cmp(big.NewInt(int64(liquidityTier.BasePositionN
// 기본 포지션 크기를 초과하는 부분에 대해 추가 마진 적용
excessNotional := new(big.Int).Sub(
bigAbsNotionalQuoteQuantums,
big.NewInt(int64(liquidityTier.BasePositionNotional)),
)
// 임팩트 기반 추가 마진 계산
additionalMarginFactor := calculateImpactMarginFactor(excessNotional, liquidityT
additionalInitialMargin := lib.BigIntMulPpm(excessNotional, additionalMarginFact
additionalMaintenanceMargin := lib.BigIntMulPpm(excessNotional, additionalMargin
bigInitialMarginQuoteQuantums.Add(bigInitialMarginQuoteQuantums, additionalIniti
bigMaintenanceMarginQuoteQuantums.Add(bigMaintenanceMarginQuoteQuantums, additio
}
return  bigInitialMarginQuoteQuantums, bigMaintenanceMarginQuoteQuantums
}
```

3. 청산 조건 판단

청산 가능 여부 확인

```
func CanLiquidateSubaccount(bigNetCollateral *big.Int, bigMaintenanceMargin *bi
// 청산 조건:
// 1. 유지 마진 요구사항이 0보다 커야 함 (포지션이 있어야 함)
// 2. 순 담보가 유지 마진 요구사항보다 작아야 함
return bigMaintenanceMargin.Sign() > 0 && bigMaintenanceMargin.Cmp(bigNetCollat
}

func (k Keeper) CheckSubaccountLiquidation(
ctx sdk.Context,
subaccountId types.SubaccountId,
) (isLiquidatable bool, hasNegativeTnc bool, err error) {
// 현재 서브계정 상태 조회
subaccount := k.GetSubaccount(ctx, subaccountId)
// 담보 및 마진 요구사항 계산
bigNetCollateral, _, bigMaintenanceMargin, err := k.GetNetCollateralAndMarginReq
ctx,
types.Update{SubaccountId: subaccountId},
)
if err != nil {
return false, false, err
```

```
}

// 청산 조건 확인

isLiquidatable = CanLiquidateSubaccount(bigNetCollateral, bigMaintenanceMargin)

// 음수 순 담보 확인 (Total Net Collateral < 0)

hasNegativeTnc = bigNetCollateral.Sign() == -1

return  isLiquidatable, hasNegativeTnc, nil

}
```

리스크 등급 분류

```
type  RiskLevel  int


const (

Safe  RiskLevel = iota

Warning

Danger

Liquidatable

NegativeEquity

)


func (k Keeper) GetSubaccountRiskLevel(

ctx  sdk.Context,

subaccountId  types.SubaccountId,
```

```
) (RiskLevel, error) {

bigNetCollateral, bigInitialMargin, bigMaintenanceMargin, err :=

k.GetNetCollateralAndMarginRequirements(ctx, types.Update{SubaccountId: subaccou

if err != nil {

return Safe, err

}

// 음수 순 담보

if bigNetCollateral.Sign() == -1 {

return NegativeEquity, nil

}

// 청산 가능

if CanLiquidateSubaccount(bigNetCollateral, bigMaintenanceMargin) {

return Liquidatable, nil

}

// 마진 비율 계산

if bigMaintenanceMargin.Sign() == 0 {

return Safe, nil // 포지션 없음

}

marginRatio := new(big.Float).Quo(

new(big.Float).SetInt(bigNetCollateral),

new(big.Float).SetInt(bigMaintenanceMargin),

)

// 위험 등급 분류
```

```
if marginRatio.Cmp(big.NewFloat(1.5)) >= 0 {
return Safe, nil
} else if marginRatio.Cmp(big.NewFloat(1.2)) >= 0 {
return Warning, nil
} else {
return Danger, nil
}
}
```

청산 주문 생성 및 실행 프로세스

1. 청산 대상 식별 및 우선순위

청산 대상 서브계정 수집

```
func (k Keeper) GetLiquidatableSubaccounts(
ctx sdk.Context,
) ([]types.SubaccountId, []types.SubaccountId, error) {
liquidatableSubaccounts := []types.SubaccountId{}
negativeTncSubaccounts := []types.SubaccountId{}
// 모든 서브계정 조회 (실제로는 오픈 포지션이 있는 계정만)
subaccounts := k.GetAllSubaccountsWithOpenPositions(ctx)
for _, subaccount := range subaccounts {
isLiquidatable, hasNegativeTnc, err := k.CheckSubaccountLiquidation(
```

```
ctx,
*subaccount.Id,
)
if err != nil {
continue // 에러 발생 시 건너뛰기
}
if isLiquidatable {
liquidatableSubaccounts = append(liquidatableSubaccounts, *subaccount.Id)
}
if hasNegativeTnc {
negativeTncSubaccounts = append(negativeTncSubaccounts, *subaccount.Id)
}
}
return liquidatableSubaccounts, negativeTncSubaccounts, nil
}
```

청산 우선순위 결정

```
func (k Keeper) PrioritizeLiquidations(
ctx sdk.Context,
liquidatableSubaccounts []types.SubaccountId,
) []types.SubaccountId {
type liquidationPriority struct {
subaccountId types.SubaccountId
```

```
marginRatio *big.Float
totalNotional *big.Int
}
priorities := make([]liquidationPriority, 0, len(liquidatableSubaccounts))
for _, subaccountId := range liquidatableSubaccounts {
bigNetCollateral, _, bigMaintenanceMargin, err :=
k.GetNetCollateralAndMarginRequirements(ctx, types.Update{SubaccountId: subaccou
if err != nil {
continue
}
// 마진 비율 계산 (낮을수록 우선순위 높음)
marginRatio := new(big.Float).Quo(
new(big.Float).SetInt(bigNetCollateral),
new(big.Float).SetInt(bigMaintenanceMargin),
)
// 총 포지션 크기 계산
totalNotional := k.GetTotalPositionNotional(ctx, subaccountId)
priorities = append(priorities, liquidationPriority{
subaccountId: subaccountId,
marginRatio: marginRatio,
totalNotional: totalNotional,
})
}
```

```
// 우선순위 정렬: 마진 비율이 낮고, 포지션이 큰 순서
sort.Slice(priorities, func(i, j int) bool {
ratioCompare := priorities[i].marginRatio.Cmp(priorities[j].marginRatio)
if ratioCompare == 0 {
return priorities[i].totalNotional.Cmp(priorities[j].totalNotional) > 0
}
return ratioCompare < 0
})
result := make([]types.SubaccountId, len(priorities))
for i, p := range priorities {
result[i] = p.subaccountId
}
return result
}
```

2. 청산 주문 생성

청산 주문 구조

```
type LiquidationOrder struct {
// 청산 대상 서브계정
SubaccountId types.SubaccountId
// 청산할 영구 선물 ID
PerpetualId uint32
```

```
// 청산 수량 (음수면 숏 포지션 청산, 양수면 롱 포지션 청산)
Quantums *big.Int
// 청산 가격 (파산 가격 기준)
Subticks  types.Subticks
// 청산 타입
LiquidationType  LiquidationType
}

type  LiquidationType  int

const (
// 주문서 기반 청산
OrderbookLiquidation  LiquidationType = iota
// 디레버리징 (강제 매칭)
Deleveraging
)
```

청산 주문 생성 로직

```
func (k Keeper) CreateLiquidationOrder(
ctx  sdk.Context,
subaccountId  types.SubaccountId,
perpetualId  uint32,
```

```
) (*LiquidationOrder, error) {

// 1. 현재 포지션 조회

subaccount := k.GetSubaccount(ctx, subaccountId)

position, exists := subaccount.GetPerpetualPositionForId(perpetualId)

if !exists {

return  nil, types.ErrPositionNotFound

}

// 2. 청산할 수량 결정 (전체 포지션 또는 일부)

liquidationQuantums := k.calculateLiquidationQuantums(ctx, subaccountId, positio

// 3. 파산 가격 계산

bankruptcyPrice, err := k.GetBankruptcyPriceInQuoteQuantums(

ctx,

subaccountId,

perpetualId,

liquidationQuantums,

)

if  err != nil {

return  nil, err

}

// 4. 청산 주문 생성

liquidationOrder := &LiquidationOrder{

SubaccountId: subaccountId,

PerpetualId: perpetualId,
```

```
Quantums: liquidationQuantums,

Subticks: k.convertPriceToSubticks(bankruptcyPrice, perpetualId),

LiquidationType: OrderbookLiquidation,

}

return  liquidationOrder, nil

}
```

파산 가격 계산

```
func (k Keeper) GetBankruptcyPriceInQuoteQuantums(

ctx  sdk.Context,

subaccountId  types.SubaccountId,

perpetualId  uint32,

deltaQuantums *big.Int,

) (*big.Int, error) {

// 파산 가격 공식: P_bankruptcy = (TNC + PnL_other) / |ΔQ|

// 여기서:

// - TNC: Total Net Collateral

// - PnL_other: 다른 포지션들의 미실현 손익

// - ΔQ: 청산할 포지션 수량

// 1. 총 순 담보 계산

bigNetCollateral, _, _, err := k.GetNetCollateralAndMarginRequirements(

ctx,

types.Update{SubaccountId: subaccountId},
```

```
)
if err != nil {
return nil, err
}
// 2. 현재 포지션의 미실현 손익 제외
currentPositionPnL, err := k.GetPositionPnL(ctx, subaccountId, perpetualId)
if err != nil {
return nil, err
}
adjustedCollateral := new(big.Int).Sub(bigNetCollateral, currentPositionPnL)
// 3. 파산 가격 계산
absDeltaQuantums := new(big.Int).Abs(deltaQuantums)
if absDeltaQuantums.Sign() == 0 {
return nil, types.ErrInvalidLiquidationQuantums
}
bankruptcyPrice := new(big.Int).Div(adjustedCollateral, absDeltaQuantums)
// 4. 가격 범위 검증 (시장가의 일정 범위 내)
marketPrice, err := k.pricesKeeper.GetMarketPrice(ctx, perpetualId)
if err != nil {
return nil, err
}
maxDeviation := lib.BigIntMulPpm(marketPrice.Price.BigInt(), 200000) // 20% 최대
minPrice := new(big.Int).Sub(marketPrice.Price.BigInt(), maxDeviation)
```

```
maxPrice := new(big.Int).Add(marketPrice.Price.BigInt(), maxDeviation)
if bankruptcyPrice.Cmp(minPrice) < 0 {
bankruptcyPrice = minPrice
} else if bankruptcyPrice.Cmp(maxPrice) > 0 {
bankruptcyPrice = maxPrice
}
return bankruptcyPrice, nil
}
```

3. 청산 실행 메커니즘

주문서 기반 청산

```
func (k Keeper) LiquidateSubaccountsAgainstOrderbook(
ctx sdk.Context,
liquidatableSubaccountIds []types.SubaccountId,
) ([]types.SubaccountLiquidationInfo, error) {
subaccountsToDeleverage := []types.SubaccountLiquidationInfo{}
for _, subaccountId := range liquidatableSubaccountIds {
// 각 포지션에 대해 청산 시도
subaccount := k.GetSubaccount(ctx, subaccountId)
for _, position := range subaccount.PerpetualPositions {
if position.GetBigQuantums().Sign() == 0 {
continue // 포지션이 없으면 건너뛰기
```

```
}
// 청산 주문 생성
liquidationOrder, err := k.CreateLiquidationOrder(
ctx,
subaccountId,
position.PerpetualId,
)
if  err != nil {
continue
}
// MemCLOB에서 매칭 시도
_, orderStatus, _, err := k.MemClob.PlaceOrder(ctx, liquidationOrder)
if  err != nil || orderStatus != types.Success {
// 주문서에서 매칭 실패 시 디레버리징 대상에 추가
subaccountsToDeleverage = append(subaccountsToDeleverage,
types.SubaccountLiquidationInfo{
SubaccountId: subaccountId,
PerpetualId: position.PerpetualId,
Quantums: position.GetBigQuantums(),
})
}
}
}
```

```
return subaccountsToDeleverage, nil
}
```

디레버리징 (강제 매칭)

```
func (k Keeper) DeleverageSubaccounts(
ctx sdk.Context,
subaccountsToDeleverage []types.SubaccountLiquidationInfo,
) error {
for _, liquidationInfo := range subaccountsToDeleverage {
// 반대 포지션을 가진 계정들 찾기
counterparties, err := k.FindDeleveragingCounterparties(
ctx,
liquidationInfo.PerpetualId,
liquidationInfo.Quantums,
)
if err != nil {
continue
}
// 강제 매칭 실행
err = k.ExecuteDeleveraging(ctx, liquidationInfo, counterparties)
if err != nil {
// 디레버리징 실패 시 보험 기금 사용
```

```
err = k.ProcessInsuranceFundLoss(ctx, liquidationInfo)

if err != nil {

// 보험 기금도 부족한 경우 사회화 손실

err = k.SocializeLoss(ctx, liquidationInfo)

if err != nil {

return fmt.Errorf("failed to handle liquidation loss: %w", err)

}

}

}

}

return nil

}
```

디레버리징 상대방 선택

```
func (k Keeper) FindDeleveragingCounterparties(

ctx sdk.Context,

perpetualId uint32,

liquidationQuantums *big.Int,

) ([]types.DeleveragingCounterparty, error) {

// 반대 포지션을 가진 계정들 조회

isLiquidationLong := liquidationQuantums.Sign() > 0

targetSide := !isLiquidationLong // 반대 사이드

candidates := k.GetSubaccountsWithPositions(ctx, perpetualId, targetSide)
```

```
// 수익률 기준으로 정렬 (높은 수익률 우선)
sort.Slice(candidates, func(i, j int) bool {
pnlI := k.GetPositionPnL(ctx, candidates[i].SubaccountId, perpetualId)
pnlJ := k.GetPositionPnL(ctx, candidates[j].SubaccountId, perpetualId)
return pnlI.Cmp(pnlJ) > 0
})
counterparties := []types.DeleveragingCounterparty{}
remainingQuantums := new(big.Int).Abs(liquidationQuantums)
for _, candidate := range candidates {
if remainingQuantums.Sign() == 0 {
break
}
// 매칭할 수량 계산
availableQuantums := new(big.Int).Abs(candidate.Position.GetBigQuantums())
matchQuantums := lib.BigIntMin(remainingQuantums, availableQuantums)
counterparties = append(counterparties, types.DeleveragingCounterparty{
SubaccountId: candidate.SubaccountId,
Quantums: matchQuantums,
})
remainingQuantums.Sub(remainingQuantums, matchQuantums)
}
return counterparties, nil
```

```
}
```

보험 기금 운용 및 손실 처리 메커니즘

1. 보험 기금 구조

보험 기금 타입

```
type InsuranceFund struct {
// 크로스 마진 보험 기금 (모든 크로스 마진 시장 공유)
CrossMarginFund *big.Int
// 격리 마진 보험 기금 (시장별 독립)
IsolatedMarginFunds map[uint32]*big.Int // perpetualId -> fund balance
}

func (k Keeper) GetInsuranceFundBalance(
ctx sdk.Context,
perpetualId uint32,
) (*big.Int, error) {
perpetual, err := k.GetPerpetual(ctx, perpetualId)
if err != nil {
return nil, err
}
// 보험 기금 모듈 주소 조회
```

```
insuranceFundAddr, err := k.GetInsuranceFundModuleAddress(ctx, perpetualId)

if err != nil {

return nil, err

}

// USDC 잔고 조회

balance := k.bankKeeper.GetBalance(ctx, insuranceFundAddr, constants.Usdc.Denom)

return balance.Amount.BigInt(), nil

}
```

보험 기금 모듈 주소 관리

```
func (k Keeper) GetInsuranceFundModuleAddress(

ctx sdk.Context,

perpetualId uint32,

) (sdk.AccAddress, error) {

perpetual, err := k.GetPerpetual(ctx, perpetualId)

if err != nil {

return nil, err

}

var moduleName string

if perpetual.Params.MarketType == types.PerpetualMarketType_PERPETUAL_MARKET_TY

// 격리 마진: 시장별 독립 보험 기금

moduleName = fmt.Sprintf("%s:%d", types.InsuranceFundName, perpetualId)

} else {
```

```
// 크로스 마진: 공통 보험 기금
moduleName = types.InsuranceFundName
}
return k.accountKeeper.GetModuleAddress(moduleName), nil
}
```

2. 손실 처리 프로세스

보험 기금을 통한 손실 보전

```
func (k Keeper) ProcessInsuranceFundLoss(
ctx sdk.Context,
liquidationInfo types.SubaccountLiquidationInfo,
) error {
// 1. 청산으로 인한 손실 계산
loss, err := k.CalculateLiquidationLoss(ctx, liquidationInfo)
if err != nil {
return err
}
if loss.Sign() <= 0 {
return nil // 손실이 없거나 이익인 경우
}
// 2. 보험 기금 잔고 확인
```

```
insuranceFundBalance, err := k.GetInsuranceFundBalance(ctx, liquidationInfo.Perp

if err != nil {

return err

}

// 3. 보험 기금으로 손실 보전

if insuranceFundBalance.Cmp(loss) >= 0 {

// 보험 기금이 충분한 경우

err = k.TransferInsuranceFundPayments(ctx, loss, liquidationInfo.PerpetualId)

if err != nil {

return err

}

// 청산 완료 처리

return k.CompleteLiquidation(ctx, liquidationInfo, loss)

} else {

// 보험 기금이 부족한 경우

if insuranceFundBalance.Sign() > 0 {

// 보험 기금의 모든 잔고 사용

err = k.TransferInsuranceFundPayments(ctx, insuranceFundBalance, liquidationInfo

if err != nil {

return err

}

}

// 남은 손실은 사회화
```

```
remainingLoss := new(big.Int).Sub(loss, insuranceFundBalance)

return k.SocializeLoss(ctx, liquidationInfo, remainingLoss)

}

}
```

보험 기금 이전

```
func (k Keeper) TransferInsuranceFundPayments(

ctx sdk.Context,

insuranceFundDelta *big.Int,

perpetualId uint32,

) error {

if insuranceFundDelta.Sign() == 0 {

return nil

}

// USDC 코인으로 변환

coin, err := k.assetsKeeper.ConvertAssetToCoin(

ctx,

assettypes.AssetUsdc.Id,

new(big.Int).Abs(insuranceFundDelta),

)

if err != nil {

return err
```

```
}

// 모듈 주소 조회

subaccountsModule := k.accountKeeper.GetModuleAddress(types.ModuleName)

insuranceFundAddr, err := k.GetInsuranceFundModuleAddress(ctx, perpetualId)

if err != nil {

return err

}

// 이전 방향 결정

var fromModule, toModule sdk.AccAddress

if insuranceFundDelta.Sign() < 0 {

// 보험 기금 → 서브계정 모듈 (손실 보전)

fromModule, toModule = insuranceFundAddr, subaccountsModule

} else {

// 서브계정 모듈 → 보험 기금 (수익 적립)

fromModule, toModule = subaccountsModule, insuranceFundAddr

}

// 코인 이전 실행

return k.bankKeeper.SendCoins(ctx, fromModule, toModule, sdk.Coins{coin})

}
```

3. 사회화 손실 (Socialized Loss)

손실 분배 알고리즘

```
func (k Keeper) SocializeLoss(
ctx  sdk.Context,
liquidationInfo  types.SubaccountLiquidationInfo,
loss *big.Int,
) error {
// 1. 같은 시장의 수익 포지션을 가진 계정들 조회
profitableAccounts := k.GetProfitableAccounts(ctx, liquidationInfo.PerpetualId)
if  len(profitableAccounts) == 0 {
return  types.ErrNoAccountsForSocialization
}
// 2. 총 수익 계산
totalProfit := big.NewInt(0)
for  _, account := range  profitableAccounts {
profit := k.GetPositionPnL(ctx, account.SubaccountId, liquidationInfo.PerpetualI
if  profit.Sign() > 0 {
totalProfit.Add(totalProfit, profit)
}
}
if  totalProfit.Sign() <= 0 {
return  types.ErrInsufficientProfitForSocialization
}
// 3. 손실 비례 분배
```

```
updates := []types.Update{}

remainingLoss := new(big.Int).Set(loss)

for  i, account := range  profitableAccounts {

profit := k.GetPositionPnL(ctx, account.SubaccountId, liquidationInfo.PerpetualI

if  profit.Sign() <= 0 {

continue

}

var  lossShare *big.Int

if  i == len(profitableAccounts)-1 {

// 마지막 계정은 남은 손실 모두 부담 (반올림 오차 처리)

lossShare = remainingLoss

} else {

// 수익 비례로 손실 분배

lossShare = new(big.Int).Div(

new(big.Int).Mul(loss, profit),

totalProfit,

)

remainingLoss.Sub(remainingLoss, lossShare)

}

// 최대 수익 범위 내에서만 손실 부담

if  lossShare.Cmp(profit) > 0 {

lossShare = profit

}
```

```
// 서브계정 업데이트 생성
update := types.Update{
SubaccountId: account.SubaccountId,
AssetUpdates: []types.AssetUpdate{
{
AssetId: assettypes.AssetUsdc.Id,
BigQuantumsDelta: new(big.Int).Neg(lossShare),
},
},
}
updates = append(updates, update)
}
// 4. 일괄 업데이트 실행
success, _, err := k.UpdateSubaccounts(ctx, updates, types.SocializedLoss)
if !success || err != nil {
return  fmt.Errorf("failed to socialize loss: %w", err)
}
// 5. 사회화 손실 이벤트 생성
ctx.EventManager().EmitEvent(
types.NewSocializedLossEvent(
liquidationInfo.PerpetualId,
loss,
len(updates),
```

```
),
)
return nil
}
```

수익 계정 식별

```
func (k Keeper) GetProfitableAccounts(
ctx sdk.Context,
perpetualId uint32,
) []types.ProfitableAccount {
profitableAccounts := []types.ProfitableAccount{}
// 해당 시장에 포지션을 가진 모든 계정 조회
subaccounts := k.GetSubaccountsWithPositions(ctx, perpetualId)
for _, subaccount := range subaccounts {
// 포지션의 미실현 손익 계산
pnl := k.GetPositionPnL(ctx, *subaccount.Id, perpetualId)
if pnl.Sign() > 0 {
profitableAccounts = append(profitableAccounts, types.ProfitableAccount{
SubaccountId: *subaccount.Id,
Profit: pnl,
})
}
}
```

```
// 수익 크기 순으로 정렬 (큰 수익부터)
sort.Slice(profitableAccounts, func(i, j int) bool {
return profitableAccounts[i].Profit.Cmp(profitableAccounts[j].Profit) > 0
})
return profitableAccounts
}
```

4. 시스템 안정성 메커니즘

청산 한도 관리

```
type LiquidationConfig struct {
// 블록당 최대 청산 금액 (서브계정별)
MaxLiquidationFeePpm uint32
// 블록당 최대 보험 기금 손실
MaxInsuranceFundLossPerBlock *big.Int
// 청산 수수료율
LiquidationFeePpm uint32
// 디레버리징 수수료율
DeleveragingFeePpm uint32
}

func (k Keeper) CheckLiquidationLimits(
ctx sdk.Context,
```

```
subaccountId  types.SubaccountId,
liquidationAmount *big.Int,
) error {
config := k.GetLiquidationConfig(ctx)
// 1. 서브계정별 청산 한도 확인
blockHeight := ctx.BlockHeight()
currentBlockLiquidation := k.GetSubaccountLiquidationInBlock(ctx, subaccountId,
if  new(big.Int).Add(currentBlockLiquidation, liquidationAmount).Cmp(
config.MaxLiquidationFeePpm) > 0 {
return  types.ErrLiquidationExceedsLimit
}
// 2. 보험 기금 손실 한도 확인
currentBlockInsuranceLoss := k.GetInsuranceFundLossInBlock(ctx, blockHeight)
if  currentBlockInsuranceLoss.Cmp(config.MaxInsuranceFundLossPerBlock) > 0 {
return  types.ErrInsuranceFundLossExceedsLimit
}
return  nil
}
```

시스템 건전성 모니터링

```
func (k Keeper) GetSystemHealthMetrics(ctx  sdk.Context) types.SystemHealthMetri
metrics := types.SystemHealthMetrics{}
```

```
// 1. 총 보험 기금 잔고
metrics.TotalInsuranceFundBalance = k.GetTotalInsuranceFundBalance(ctx)
// 2. 청산 대상 계정 수
liquidatableAccounts, _ := k.GetLiquidatableSubaccounts(ctx)
metrics.LiquidatableAccountsCount = len(liquidatableAccounts)
// 3. 시스템 총 순 담보
metrics.SystemNetCollateral = k.GetSystemNetCollateral(ctx)
// 4. 평균 마진 비율
metrics.AverageMarginRatio = k.GetAverageMarginRatio(ctx)
// 5. 최근 24시간 청산 통계
metrics.RecentLiquidationStats = k.GetRecentLiquidationStats(ctx, 24*time.Hour)
return metrics
}
```

성능 최적화 및 확장성

1. 청산 처리 최적화

배치 청산 처리

```
func (k Keeper) BatchProcessLiquidations(
ctx sdk.Context,
liquidatableSubaccounts []types.SubaccountId,
maxBatchSize int,
```

```
) error {
// 배치 크기로 분할 처리
for i := 0; i < len(liquidatableSubaccounts); i += maxBatchSize {
end := i + maxBatchSize
if end > len(liquidatableSubaccounts) {
end = len(liquidatableSubaccounts)
}
batch := liquidatableSubaccounts[i:end]
// 배치 단위로 청산 처리
err := k.ProcessLiquidationBatch(ctx, batch)
if err != nil {
// 개별 계정별로 재시도
for _, subaccountId := range batch {
k.ProcessSingleLiquidation(ctx, subaccountId)
}
}
}
return nil
}
```

2. 메모리 및 계산 최적화

캐싱 전략

```go
type LiquidationCache struct {
// 마진 계산 결과 캐시
marginCache map[string]*MarginCalculation
// 가격 데이터 캐시
priceCache map[uint32]*big.Int
// 캐시 만료 시간
cacheExpiry time.Time
}


func (k Keeper) GetCachedMarginCalculation(
ctx sdk.Context,
subaccountId types.SubaccountId,
) (*MarginCalculation, bool) {
cache := k.getLiquidationCache(ctx)
if cache.cacheExpiry.Before(ctx.BlockTime()) {
// 캐시 만료
return nil, false
}
key := fmt.Sprintf("%s_%d", subaccountId.String(), ctx.BlockHeight())
if calc, exists := cache.marginCache[key]; exists {
return calc, true
}
```

```
    return  nil, false

}
```

결론

dYdX v4의 리스크 관리 및 청산 시스템은 다층적이고 포괄적인 접근 방식을 통해 시스템의 안정성과 사용자 자산의 보호를 달성합니다. 주요 특징은 다음과 같습니다:

1. **정밀한 리스크 계산**: 유동성 계층별 차등 마진 정책과 실시간 담보 모니터링
2. **다단계 청산 메커니즘**: 주문서 청산 → 디레버리징 → 보험 기금 → 사회화 손실
3. **견고한 손실 처리**: 보험 기금과 사회화 손실을 통한 시스템 건전성 유지
4. **성능 최적화**: 배치 처리와 캐싱을 통한 효율적인 대규모 청산 처리
5. **투명한 모니터링**: 실시간 시스템 건전성 메트릭과 청산 통계 제공

이러한 설계를 통해 dYdX v4는 높은 레버리지 거래 환경에서도 시스템 리스크를 효과적으로 관리하고, 극단적인 시장 상황에서도 플랫폼의 안정성을 유지할 수 있습니다.